Case Study II: Mechanism Design for Robot Swarms **DUE:** 9:00AM, Nov. 2, 2026

Background reading: Chapter 9 (linear dynamical systems); course notes on matrix–vector and matrix–matrix products.

A. Introduction

A.1 The design problem

A search team drops a swarm of small robots by air into open terrain in a search and rescue scenario. No robot has a map, a global view, or a connection to a central computer. Each robot can do only two things: talk to the robots within radio range, and move. Yet the team needs the swarm to act as one, for example by gathering at a rally point.

This is a decentralized control engieering problem, related to the notion of mechanism design. The design goal is to choose the rules of a system so that when each participant follows its own local rule, the system as a whole produces a desired outcome. In economics, mechanism design concerns people responding to incentives. With robots, the designer writes the agents' rules directly, but still cannot see or command the whole swarm at once. Every global behavior must emerge from local interactions.

This Case Study provides an initial introduction to many high-level areas in ESE, including Control, Communication, and Optimization.

A.2 The mechanism is a matrix

Robot i is at position (x_i, y_i). Collect every robot's x-coordinate into a vector $\mathbf{x} \in \mathbb{R}^n$, and every y-coordinate into $\mathbf{y} \in \mathbb{R}^n$. Each robot follows the same local rule in both directions, so the swarm is a discrete-time linear system applied to each coordinate:

$$\mathbf{x}[k+1] = W\mathbf{x}[k], \qquad \mathbf{y}[k+1] = W\mathbf{y}[k] \qquad \text{(later, plus an input term).}$$

The *mechanism* is the matrix W: entry W_{ij} says how strongly robot i moves toward robot j. Your tools throughout are the two operations at the core of this course: the **matrix–vector product** (one step of the swarm) and the **matrix–matrix product** (composing steps, and building the network itself).

A.3 Roadmap

Part I. Build networks. Construct four communication networks and measure their structure using matrix products.

Part II. Run dynamics. Program two rendezvous protocols and explain where and how fast the swarm gathers.

Part III. Design. Write a strategy that chooses which robots should lead, for deployments you have never seen, and enter it in a class-wide competition.

A.4 Ground rules for code

- Write all simulation and analysis code yourself. Do not use `lsim`, `pdist`, or MATLAB's built-in graph analysis functions. You may use `plot` and `graph` only to *visualize*.
- Test every function on a small example whose answer you can check by eye (for example, a 5-robot star or ring), and show these tests in your published output.
- You are responsible for being able to defend every line of code you turn in.

B. Part I: Communication Networks

B.1 Adjacency matrices

The swarm's communication network is an $n \times n$ **adjacency matrix** G, with $G_{ij} = 1$ if robots i and j can communicate and $G_{ij} = 0$ otherwise. All links are two-way ($G = G^\top$) and no robot links to itself ($G_{ii} = 0$). Let $\mathbf{e}_i$ be the ith standard basis vector and $\mathbf{1}$ the all-ones vector.

B.2 Deployment

Throughout Parts I and II, $n = 100$ robots are dropped into two 5×5 zones that are 1 unit apart:

```
rng(105); P0 = [5*rand(50,2); 5*rand(50,2) + [6 0]];
```

Robots 1–50 land in the left zone and 51–100 in the right zone. Row i of `P0` is robot i's starting position $\mathbf{p}_i$, and its columns are $\mathbf{x}[0]$ and $\mathbf{y}[0]$. Nothing blocks the robots' movement.

Modeling assumption. Each robot keeps the communication partners it had when it landed. The network G is computed once, from `P0`, and does not change as the robots move.

B.3 Four networks

Where a matrix expression is given, your code must build G *from that expression*, not by looping over entries.

T1. Broadcast (complete). Every robot hears every other robot: $G = \mathbf{1}\mathbf{1}^\top - I$.

T2. Base station (star). Robot 1 is a hub that relays for everyone; no other links exist:

$$G = \mathbf{e}_1\mathbf{1}^\top + \mathbf{1}\mathbf{e}_1^\top - 2\,\mathbf{e}_1\mathbf{e}_1^\top.$$

T3. Ring. Robots are assigned to a ring, and each talks to its k nearest ring-neighbors on each side. With S the cyclic shift matrix, defined by $S\mathbf{e}_j = \mathbf{e}_{j+1}$ (indices mod n),

$$G = \sum_{j=1}^{k} \left(S^j + (S^j)^\top \right).$$

T4. Radio range (disk graph). Robots $i \neq j$ can communicate if they are within distance r of each other. With P the $n \times 2$ matrix of positions and $\mathbf{s}$ the vector with entries $s_i = x_i^2 + y_i^2$, the matrix of squared distances between all pairs of robots is

$$\Delta = \mathbf{s}\mathbf{1}^\top + \mathbf{1}\mathbf{s}^\top - 2\,PP^\top.$$

Form G by comparing Δ to r^2, then remove the diagonal.

Choosing parameters. For T4, find the smallest radius $r_{\min}$ (to the nearest 0.1) for which the network is connected, using your diameter function from B.4, below. Then choose the ring's k so that T3's edge count is as close as possible to T4's. This makes the comparison of T3 and T4 about *structure*, not number of links.

Code you will write

`make_complete(n)`, `make_star(n)`, `make_ring(n,k)`, `make_disk(P,r)`: each returns a symmetric 0/1 adjacency matrix with zero diagonal.

B.4 Structure from matrix products

Quantity	Formula	Meaning
Degree vector	$\mathbf{d} = G\mathbf{1}$	number of robots each robot can hear
Edge count	$\frac{1}{2}\,\mathbf{1}^\top G\mathbf{1}$	total number of links
Diameter	smallest k with $(I+G)^k$ entrywise positive	most relay hops needed between any two robots

The entries of $(I+G)^k$ grow very large, so for the diameter track only *which* entries are nonzero. Start from $R = I + G$ and repeatedly update $R \leftarrow \big((I+G)R > 0\big)$, converted back to a numeric 0/1 matrix. Stop when no zeros remain, or after n steps, in which case the network is disconnected.

Code you will write

`stats = graph_stats(G)`: returns the degree vector, edge count, and diameter (`Inf` if disconnected).

Deliverable. A table comparing T1–T4 on edge count, mean degree, maximum degree, and diameter, and a plot of the T4 network drawn over the robots' positions.

C. Part II: Rendezvous Protocols

C.1 Two protocols

The swarm's first task is **rendezvous**: every robot should gather at a common point, using only what it hears from its neighbors. You will compare two protocols. With $D = \mathrm{diag}(\mathbf{d})$ and the

Laplacian $L = D - G$:

$$\textbf{Protocol A (neighbor averaging):} \quad W_A = (I + D)^{-1}(I + G),$$
$$\textbf{Protocol B (step toward neighbors):} \quad W_B = I - \varepsilon L, \quad \varepsilon = 1/\max_i d_i.$$

In Protocol A, each robot moves to the average position of itself and its neighbors. The operation $(H)^{-1}$ is known as a matrix inverse. We will learn about this soon. In the meantime, you can consider it to be a generation of a scalar inverse, and compute it in MATLAB using the syntax `1/H`. Be careful with the order of operations here. In Protocol B, each robot takes a small step toward each of its neighbors. Either way, $\mathbf{x}[k+1] = W\mathbf{x}[k]$ and $\mathbf{y}[k+1] = W\mathbf{y}[k]$, with the same W.

Checkpoint (by hand – include in report appendix)

(a) For each protocol, write the ith entry of $\mathbf{x}[k+1] = W\mathbf{x}[k]$ as an equation for $x_i[k+1]$, and interpret it.

(b) Show that $W_A\mathbf{1} = \mathbf{1}$ and $W_B\mathbf{1} = \mathbf{1}$. What happens if all robots are already at the same point?

(c) Let $\mathbf{v} = (I + D)\mathbf{1}$. Using only $G = G^\top$ and properties of matrix products, show that $\mathbf{v}^\top W_A = \mathbf{v}^\top$ and $\mathbf{1}^\top W_B = \mathbf{1}^\top$. For each protocol, what quantity stays constant as $\mathbf{x}[k]$ evolves?

(d) Show that with $\varepsilon = 1/\max_i d_i$, every entry of W_B is nonnegative. What could go wrong with a larger ε?

Code you will write

`X = simulate_linear(W, x0, K)`: returns an $n \times (K+1)$ matrix whose column $k+1$ is $\mathbf{x}[k]$, computed with a loop of matrix–vector products. Check that the final column equals `W^K * x0`. Call it once for each coordinate.

C.2 Experiments

Use the networks from Part I and the starting positions `P0` throughout. Measure how spread out the swarm is with

$$\delta[k] = \sqrt{\|C\mathbf{x}[k]\|^2 + \|C\mathbf{y}[k]\|^2}, \qquad C = I - \tfrac{1}{n}\mathbf{1}\mathbf{1}^\top,$$

where multiplying by C subtracts the average from a vector. The **rendezvous time** k_ε is the first k with $\delta[k] \le 0.01\,\delta[0]$.

Prediction log. Before running each experiment, record a one- or two-sentence prediction. Afterward, note whether you were right and explain any surprise. You are graded on the explanation, not on whether the prediction was correct.

E1. How fast? Run Protocol A on T1–T4. Plot $\delta[k]$ on a log scale, all on one figure, and report k_ε for each. Repeat T4 with $r = r_{\min} + 0.4$. Rank the networks and explain the ranking using your Part I table. What does the gap between the two zones do?

E2. Where do they meet? Using part (c) of the Checkpoint, predict the rendezvous point for each protocol as a formula in $\mathbf{x}[0]$ and $\mathbf{y}[0]$. Verify your formulas numerically on every network. On the star, how far from the swarm's average position does each protocol meet, and why? Which protocol reaches rendezvous faster on the star, and why?

D. Part III: Mission Design and Competition

D.1 Leaders and faulty robots

Some robots do not simply follow their neighbors. Give each robot i a **responsiveness** $\lambda_i \in [0, 1]$ and an **anchor** position (a_i, b_i), collected into vectors $\mathbf{a}$ and $\mathbf{b}$. Then

$$\mathbf{x}[k+1] = \Lambda W \mathbf{x}[k] + (I - \Lambda)\mathbf{a}, \qquad \mathbf{y}[k+1] = \Lambda W \mathbf{y}[k] + (I - \Lambda)\mathbf{b}, \qquad \Lambda = \mathrm{diag}(\lambda_1, \ldots, \lambda_n).$$

Each robot moves partly according to the protocol (weight λ_i) and partly toward its anchor (weight $1 - \lambda_i$).

- **Followers** have $\lambda_i = 1$.
- **Leaders** know the rally point $\mathbf{g}$: they have $\lambda_i = 0.5$ and anchor $\mathbf{g}$.
- **Faulty robots** have a broken drive: $\lambda_i = 0$, and their anchor is where they landed. They still broadcast that position to their neighbors.

Use $W = W_A$ throughout Part III.

- Write `simulate_anchored(W, lambda, a, x0, K)`, and call it once per coordinate.
- A steady state (or, equilibrium) $\mathbf{x}^*$ must satisfy $\mathbf{x}^* = \Lambda W \mathbf{x}^* + (I - \Lambda)\mathbf{a}$ (why?). Rearrange this into a linear system, solve it with backslash (and likewise for $\mathbf{y}^*$), and confirm it matches your long-run simulation.
- On a practice deployment (D.2) with no leaders, where does the swarm end up, and why? Use the steady state to explain.

D.2 The challenge

In a real deployment, nobody knows in advance where the robots will land. So instead of choosing leaders for one particular swarm, you will design a **leader-selection strategy**: a function that looks at *any* deployment and decides which robots should lead.

```
S = choose_leaders(G, P0, faulty, g, m)
```

Given the communication network `G`, landing positions `P0`, the indices of the `faulty` robots, the rally point `g`, and a budget `m`, it returns the indices of at most `m` working robots to make leaders. Leaders use $\lambda_i = 0.5$ and anchor $\mathbf{g}$, as in D.1.

Practice deployments. The provided function `D = make_deployment(seed)` generates a random deployment, stored as a struct with fields `G`, `P0`, `r`, `faulty`, and `g`: 150 robots dropped into 2–4 zones of varying size and spacing, a radio range large enough to connect them, a few faulty robots, and a rally point. Each `seed` gives a different deployment. Use this as many times as you like. As a check on your Part I code, `make_disk(D.P0, D.r)` should reproduce `D.G` exactly.

Competition. Your `choose_leaders` will be run on up to 10 hidden deployments with `m` = 4. The hidden deployments come from the same generator with seeds you have not seen.

D.3 Scoring

For one deployment, let $e[k]$ be the mean squared distance from the working robots to $\mathbf{g}$ at step k, for the baseline with no leaders (e_{base}) and with your leaders (e_{new}), for $k = 1, \ldots, T$ with $T = 125$. The score for that deployment is

$$J = \frac{100}{T} \sum_{k=1}^{T} \left(1 - \frac{e_{\text{new}}[k]}{e_{\text{base}}[k]} \right) \qquad \text{(average percent reduction in distance to } \mathbf{g}\text{)}.$$

Because the benefit is averaged over every step, *how fast* the swarm arrives matters, not only where it ends up. Your competition score is the average of J over the hidden deployments.

Code you will write

`J = score_mission(D, S)`: computes J for deployment `D` and leader set `S`, and checks that `S` contains at most `m` distinct working robots. Use it to evaluate strategies on practice deployments. The instructor's implementation is authoritative for the competition.

D.4 Rules for `choose_leaders`

- It may use only the network's structure: anything you can compute from `G`, `P0`, `faulty`, and `g` with the tools from Part I (e.g., degrees, hop counts, distances, products and powers of G, and so on).
- It may **not** simulate the swarm or call `score_mission`. A strategy that tries candidates by simulation is a search, not a design. Your code will be reviewed for this. The competition may use values of λ_i and T slightly different from the practice values, so a strategy tuned to one exact setting will not succeed.
- It must run in under 2 seconds on one deployment.
- Start from the provided stub `choose_leaders.m`; do not change its name or the order of its inputs, and put any helper functions in the same file.

D.5 Steps

Step 1. Explore. Plot several practice deployments. Then evaluate three simple strategies on at least 20 practice deployments: random working robots, the highest-degree robots, and the robots closest to $\mathbf{g}$. Make a plot comparing their score distributions. Which works best, which fails, and on what kinds of deployment? Look closely at one failure and explain it.

Step 2. Design. Using what you learned in Part II and Step 1, design your own strategy and implement it as `choose_leaders`.

Step 3. Evaluate and justify. Compare your strategy with the three simple ones on several practice deployments, reporting both average and worst-case scores. Explain *why* your strategy works in terms of the network's structure. A strategy that scores well with no explanation earns little credit; a modest strategy with a clear explanation earns a lot.

E. Report and Submission

E.1 Discussion questions

In addition to presenting your methods and results, answer:

Q1. Modeling. We held the network fixed, but real radio range depends on distance: as robots move, links appear and disappear, so W would change over time. How might that change your results?

Q2. Mechanism design. Your strategy is centralized: it looks at the whole network at once. Could each robot decide *for itself* whether to become a leader, using only what it can learn from its neighbors? What would be lost?

Q3. Safety and ethics. A few faulty robots could pull the whole swarm off course. What does that imply about a swarm in which one robot has been *deliberately* compromised? Who is accountable when a swarm's emergent behavior causes harm?

E.2 What to turn in

1. **Code**: all functions and scripts, organized by Part, with published output (`publish`) showing your tests and key results.
2. `choose_leaders.m`, following the rules in D.4. This file is used for competition scoring.
3. **Report** (6 pages, using the provided template), and your Prediction Log.
4. A signed honor code document.

E.3 Rubric

Component	Weight
Code completeness and testing: network builders, `graph_stats`, simulators, scoring	25%
Mathematical reasoning: Checkpoint; results explained through matrix operations	30%
Design and discussion: Prediction Log, strategy exploration, design and justification, Q1–Q3	25%
Presentation: clear figures; well-organized, clearly written report	20%